DAO

Desarrollo de Aplicaciones con Objetos

Python • POO • Persistencia • Interfaces • Patrones

UTN – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ciclo Lectivo 2026

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

{ }

```
class DAO:
    def aprender():
        return software
```

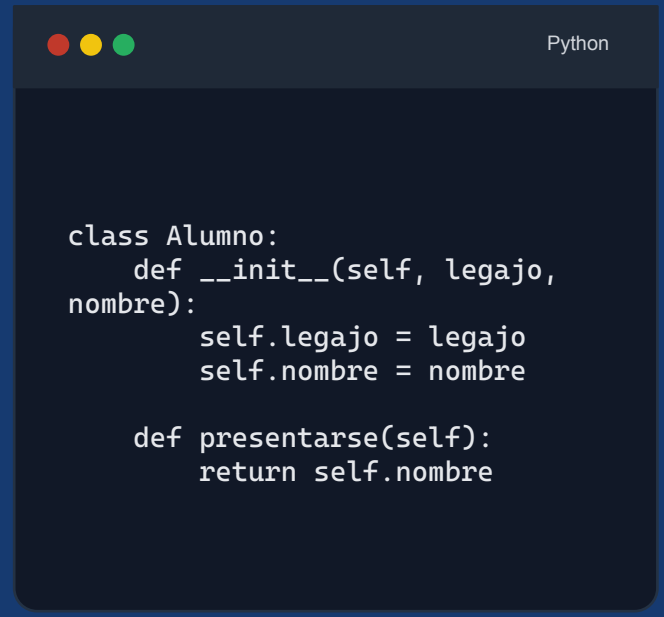
Plantilla institucional de clases

Versión 2026

DAO

Plantel Docente

Diego Javier Serrano. dserrano@frc.utn.edu.ar
Martin Gustavo Cassatti.
mcasatti@frc.utn.edu.ar
Fernando Mario Sanabria.
fsanabria@frc.utn.edu.ar
Oscar Ernesto Botta. obotta@frc.utn.edu.ar
Huenu Capdevilla. huecap7@gmail.com
Aarón Paná. aaronpana02@gmail.com

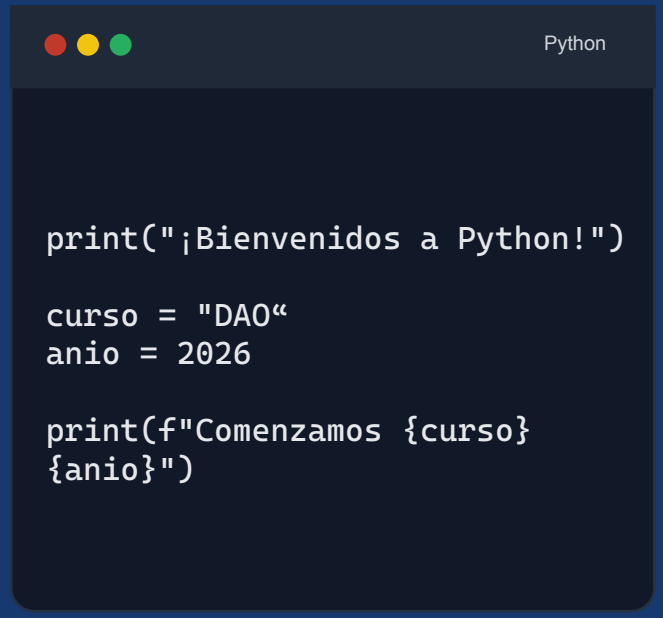


UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

DAO

Temas Clase

- Patrones de Diseño de Comportamiento
  - Strategy
  - State
  - Iterator



DAO

Material Disponible

Aula Virtual: DAO 2026

Apunte: Material de la Cátedra

https://uv.frc.utn.edu.ar/pluginfile.php/1390575/mod_resource/content/0/semana-09.pdf

Material Conceptual

https://uv.frc.utn.edu.ar/mod/resource/view.php?id=633450

Ejercicios

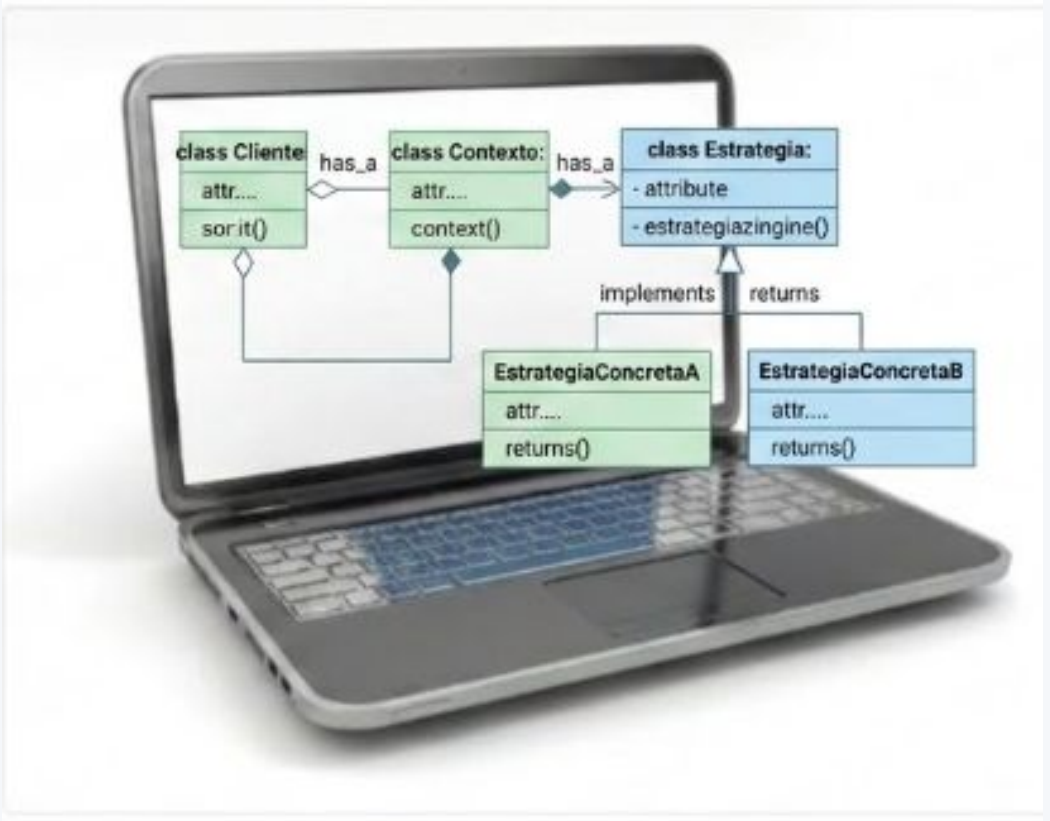
https://uv.frc.utn.edu.ar/mod/resource/view.php?id=633451



Patrones de Diseño de Comportamiento

Los patrones de diseño de comportamiento (Behavioral Design Patterns) son soluciones conocidas y reutilizables para organizar cómo se comunican los objetos, cómo se distribuyen las responsabilidades y cómo se comportan ante diferentes situaciones.

Son patrones de diseño orientados a objetos. No son clases o funciones que Python proporcione directamente, sino formas recomendadas de organizar el código para resolver problemas frecuentes.

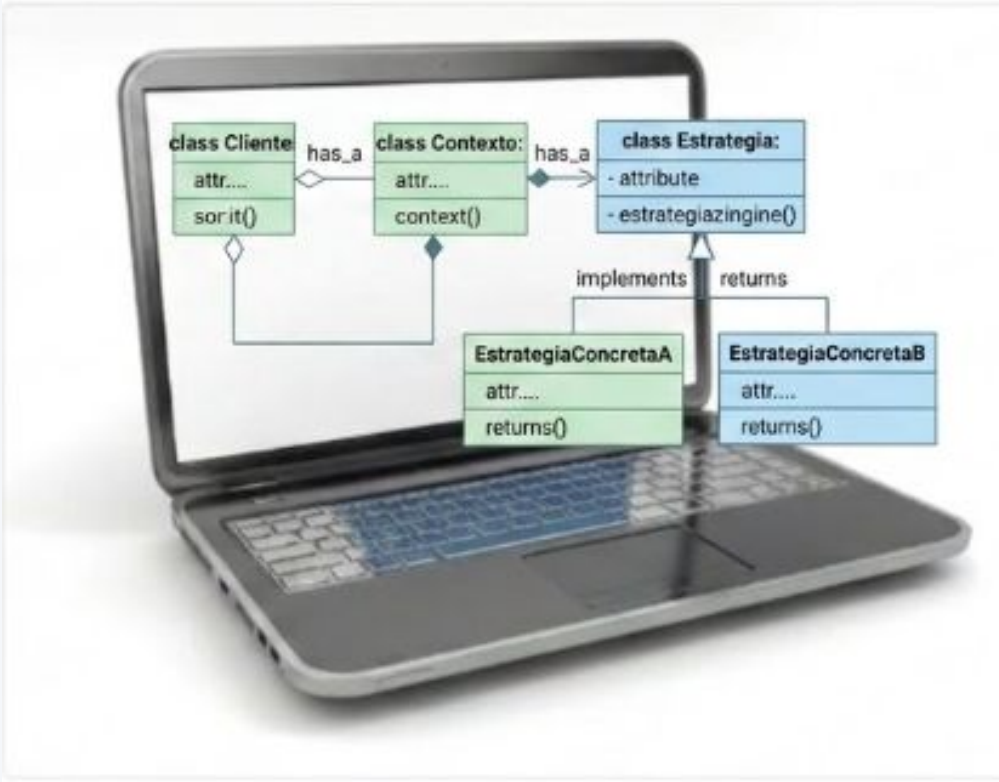


Que problemas Resuelven?

Imaginemos un sistema con varias clases que necesitan colaborar. A medida que crece, podemos encontrar problemas como:

- Muchas instrucciones `if` y `else` para elegir comportamientos.
- Clases que concentran demasiadas responsabilidades.
- Objetos muy dependientes entre sí.
- Dificultad para incorporar nuevos comportamientos sin modificar código existente.

Los patrones de comportamiento proponen distintas maneras de organizar esas responsabilidades.



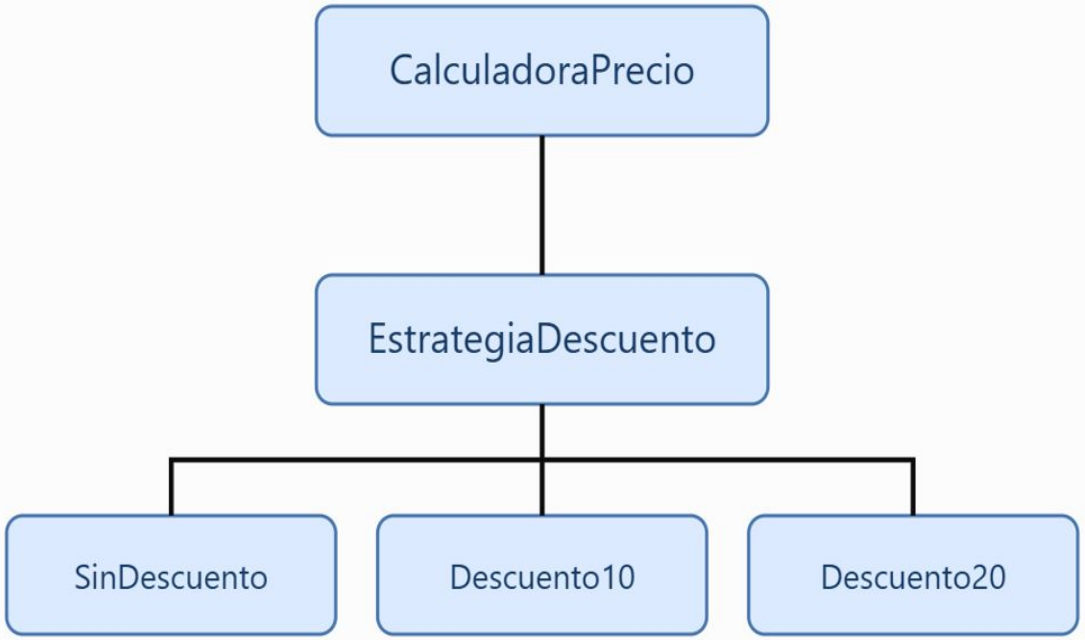
Ejemplos de Patrones

Patrón	¿Qué resuelve?	Ejemplo
Strategy	Permite cambiar el algoritmo utilizado para realizar una operación.	Calcular salarios según diferentes modalidades.
State	Permite que un objeto cambie su comportamiento según su estado actual.	Un pedido pendiente, enviado o entregado.
Iterator	Permite recorrer una colección sin conocer cómo está almacenada internamente.	Recorrer los alumnos de un curso.

Ejemplos de Patrones (II)

Patrón	Ejemplo de aplicación
Observer	Notificar a varios usuarios cuando cambia el precio de un producto.
Command	Representar acciones como guardar, copiar o deshacer mediante objetos.
Template Method	Definir los pasos generales de un proceso, permitiendo que las subclases personalicen algunos.
Chain of Responsibility	Procesar una solicitud mediante varios validadores sucesivos.
Mediator	Coordinar la comunicación entre varios componentes sin que dependan directamente entre sí.

1. Strategy (Estrategia)



La calculadora utiliza una estrategia; las estrategias concretas implementan el mismo método.

¿Qué problema resuelve?

Cuando una operación admite varias maneras de calcularse, Strategy permite encapsular cada algoritmo en una clase intercambiable. La clase que utiliza el algoritmo no necesita conocer cuál es la estrategia concreta.

Ejemplo: una calculadora de precios puede aplicar distintos descuentos. En lugar de utilizar if/elif para cada tipo de descuento, recibe un objeto estrategia.

1. Strategy (Estrategia)

1

Ejemplo Resuelto

```
from abc import ABC, abstractmethod

class EstrategiaDescuento(ABC):
    @abstractmethod
     def calcular(self, precio):
        pass

class SinDescuento(EstrategiaDescuento):
     def calcular(self, precio):
        return precio

class Descuento10(EstrategiaDescuento):
     def calcular(self, precio):
        return precio * 0.90
}
class Descuento20(EstrategiaDescuento):
     def calcular(self, precio):
        return precio * 0.80
```

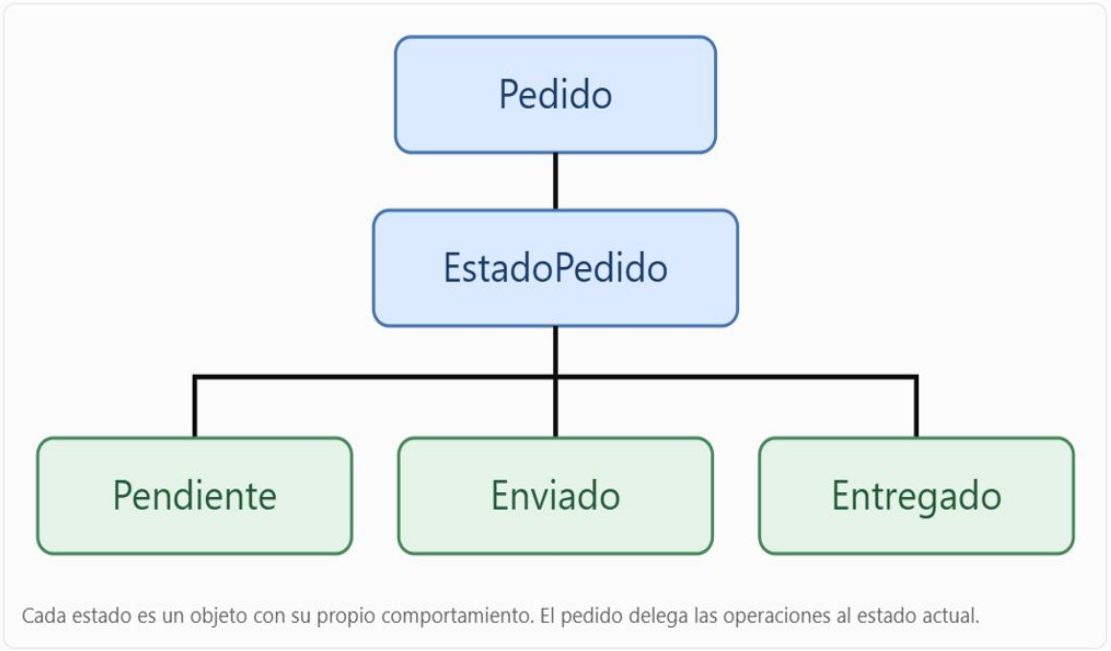
```
class CalculadoraPrecio:
     def __init__(self, estrategia):
        self.estrategia = estrategia

     def calcular_precio(self, precio):
        return self.estrategia.calcular(precio)

     def cambiar_estrategia(self, estrategia):
        self.estrategia = estrategia

calculadora = CalculadoraPrecio(Descuento10())
print(calculadora.calcular_precio(10000))  #
9000.0
calculadora.cambiar_estrategia(Descuento20())
print(calculadora.calcular_precio(10000))  #
8000.0
```

2. State (Estado)



Cada estado es un objeto con su propio comportamiento. El pedido delega las operaciones al estado actual.

¿Qué problema resuelve?

State permite que un objeto modifique su comportamiento según su estado actual. Cada estado se representa mediante un objeto con sus propias operaciones. El objeto principal delega la acción al estado vigente.

Ejemplo: un pedido pasa por Pendiente → Enviado → Entregado. Avanzar significa algo diferente en cada etapa

2. State (Estado)

2

Ejemplo Resuelto

```
from abc import ABC, abstractmethod

class EstadoPedido(ABC):
    @abstractmethod
    def avanzar(self, pedido):
        pass

class Pendiente(EstadoPedido):
    def avanzar(self, pedido):
        print("El pedido fue enviado")
        pedido.cambiar_estado(Enviado())

class Enviado(EstadoPedido):
    def avanzar(self, pedido):
        print("El pedido fue entregado")
        pedido.cambiar_estado(Entregado())
```

```
class Entregado(EstadoPedido):
    def avanzar(self, pedido):
        print("El pedido ya fue
entregado")

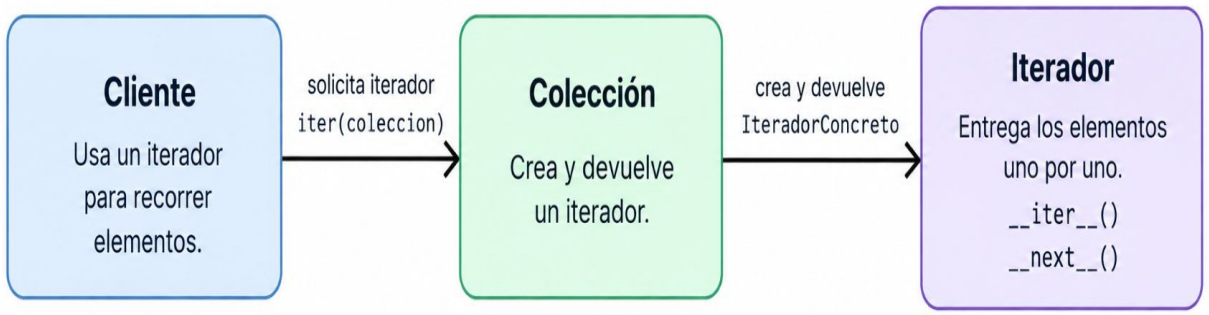
class Pedido:
    def __init__(self):
        self.estado = Pendiente()

    def cambiar_estado(self,
nuevo_estado):
        self.estado = nuevo_estado

    def avanzar(self):
        self.estado.avanzar(self)

pedido = Pedido()
pedido.avanzar()
pedido.avanzar()
pedido.avanzar()
```

3. Iterator (Iterador)



¿Qué problema resuelve?

Iterator permite recorrer una colección sin exponer cómo almacena sus elementos. En Python, los ciclos for utilizan el protocolo de iteración: iter() obtiene un iterador y next() obtiene el elemento siguiente, hasta StopIteration.

En Python ya existen iteradores, que se utilizan automáticamente en los ciclos for.

Para comprender el patrón construiremos un iterador explícito, aunque las listas de Python ya incluyen sus propios iteradores.

2. Iterator (Iterador)

3 Ejemplo Resuelto

```
class IteradorLibros:
    def __init__(self, libros):
        self.libros = libros
        self.posicion = 0

    def __iter__(self):
        return self

    def __next__(self):
       if self.posicion >= len(self.libros):
            raise StopIteration

        libro = self.libros[self.posicion]
        self.posicion += 1
        return libro
```

```
class Biblioteca:
    def __init__(self):
        self.libros = []

    def agregar_libro(self, titulo):
        self.libros.append(titulo)

    def __iter__(self):
        return IteradorLibros(self.libros)

biblioteca = Biblioteca()
biblioteca.agregar_libro("El Principito")
biblioteca.agregar_libro("Don Quijote")
biblioteca.agregar_libro("Martín Fierro")

for libro in biblioteca:
    print(libro)
```

Comparación y cierre

Patrón	Pregunta que responde	Idea clave
Strategy	¿Qué algoritmo utilizo?	Intercambiar estrategias
State	¿Qué puede hacer el objeto ahora?	Comportamiento y transición de estados
Iterator	¿Cómo recorro los elementos?	Recorrido encapsulado

State y Strategy pueden parecer similares: ambos delegan en objetos con una interfaz común. La diferencia principal es la intención: Strategy permite seleccionar un algoritmo; State representa la evolución del comportamiento de un objeto a través de sus estados.

DAO

Próximas clase

UX — conceptos generales y consultas TPI

Patrón Observer e integración de múltiples patrones

